



AWS Lambda

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Introduction to AWS Lambda



AWS Lambda is a serverless **Function as Service** that lets you run code without provisioning or managing servers.

Lambda executes your code only when needed and scales automatically to a few to a 1000 lambda functions concurrently in seconds.

You pay only for the compute time you consume. There is ***no charge when your code is not running.**

AWS Lambda Supports multiple runtimes:

- Ruby
- Python
- Java
- Go
- Powershell
- NodeJs
- C#
- Rust
- or create your own custom environment

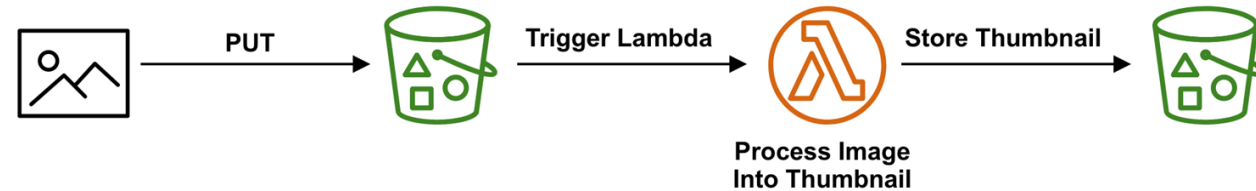


AWS Lambda - Use Cases

Lambda is commonly used to **glue different services together**, so the use cases are endless.

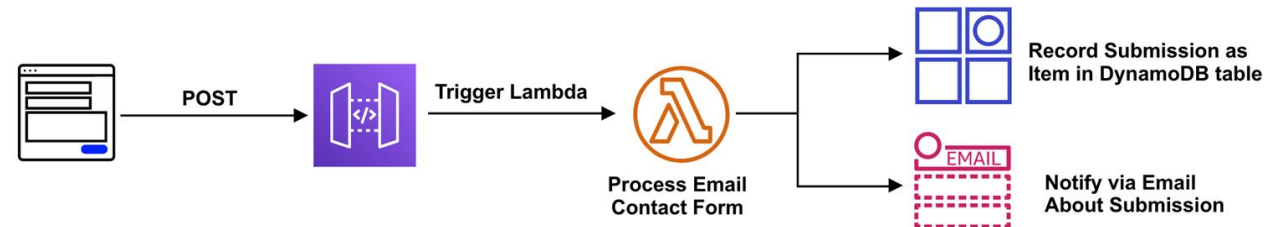
Processing Thumbnails

A web service allows users to upload their profile photos. They are stored in an S3 bucket. We can set up an Event Trigger that will invoke a Lambda, which will process the Profile Photo into a Thumbnail and store it back in the bucket.



Contact Email Form

A company has a contact email form that submits form data via API Gateway Endpoint. That endpoint triggers a lambda which validates the form data and, if valid, will save the submission in DynamoDB and send an email notification via SNS to the company



AWS Lambda - Interface

You choose your runtime

You upload your code

You choose your triggers

You grant permissions for outputs via an IAM Role

The screenshot shows the AWS Lambda console interface for a function named 'lambda_function'. The 'Code entry type' is set to 'Edit code inline' and the 'Runtime' is 'Ruby 2.5'. The handler is 'lambda_function.lambda_handler'. The code editor shows a Ruby script that uses the 'aws-sdk-firehose' gem to write records to Amazon Kinesis Firehose. The 'Triggers' section shows a trigger for 'exampro-events' (Amazon DynamoDB). The 'Permissions' section shows the function's role has access to 'Amazon CloudWatch Logs', 'Amazon DynamoDB', and 'Amazon Kinesis Firehose'. Red arrows point from the text labels to the corresponding UI elements: 'You choose your runtime' points to the 'Runtime' dropdown, 'You upload your code' points to the 'Code entry type' dropdown, 'You choose your triggers' points to the 'Add trigger' button, and 'You grant permissions for outputs via an IAM Role' points to the 'Permissions' section.

Code entry type: Edit code inline

Runtime: Ruby 2.5

Handler: lambda_function.lambda_handler

Code entry type dropdown menu:

- Edit code inline
- Edit code inline
- Upload a .zip file
- Upload a file from Amazon S3

```
1 require 'json'
2 require 'aws-sdk-firehose'
3
4 def lambda_handler(event:, context:)
5   records = []
6   event['Records'].each do |t|
7     if t['eventName'] == 'INSERT'
8       records.push({data: {
9         user_id: t['dynamodb']['NewImage']['user_id']['N'],
10        event_at: t['dynamodb']['NewImage']['event_at']['S'],
11        event_id: t['dynamodb']['NewImage']['event_id']['N'],
12        event_type: t['dynamodb']['NewImage']['event_type']['S'],
13        ip_address: t['dynamodb']['NewImage']['ip_address']['S'],
14        user_agent: t['dynamodb']['NewImage']['user_agent']['S']
15      }.to_json + "\n" })
16    end
17  end
18  json = {records_size: records.size}.to_json
19  puts json
20  unless records.size.zero?
21    firehose = Aws::Firehose::Resource.new
```

Triggers:

- exampro-events

Permissions:

- Amazon CloudWatch Logs
- Amazon DynamoDB
- Amazon Kinesis Firehose

Resources that the function's role has access to appear here

AWS Lambda – Triggers

Lambdas can be **invoked** via the AWS SDK or triggered from other AWS Services.
(This is not a complete list)

	API Gateway api application-services aws serverless
	AWS IoT aws devices iot
	Alexa Skills Kit alexa iot
	Alexa Smart Home alexa iot
	Application Load Balancer aws load-balancing
	CloudFront aws cdn edge
	CloudWatch Events aws events management-tools

	CloudWatch Logs aws logging management-tools
	CodeCommit aws developer-tools git
	Cognito Sync Trigger authentication aws identity mobile-services sync
	DynamoDB aws database nosql
	Kinesis analytics aws streaming
	S3 aws storage
	SNS aws messaging notifications pub-sub push

	SQS aws queue
---	---------------------------

AWS Lambda – Triggers

AWS Lambda can also be triggered from event sources via partner services (through event bridge)

Partner event sources (powered by Amazon EventBridge)



Datadog

Datadog is the essential monitoring platform for cloud applications.



OneLogin

OneLogin, the leader in Unified Access Management, connects people with technology through a simple and secure login, empowering organizations to access the world.



PagerDuty

PagerDuty helps AWS users automatically turn any signal into the right insight and action.



Saviynt

Saviynt enables enterprises to secure applications, data and infrastructure in a single platform for cloud and enterprise.



Segment

Segment provides the customer data infrastructure that businesses use to put their customers first.



SignalFx

SignalFx, the only real-time cloud monitoring platform for infrastructure, microservices, and applications, collects and analyzes metrics and traces across every component in your cloud environment.



SugarCRM

SugarCRM enables businesses to create extraordinary customer relationships with the most empowering, adaptable and affordable customer relationship management (CRM) solution on the market.



Whispir

Whispir is a cloud based platform that automates, personalises and layers communications using smart workflow technology.



Zendesk

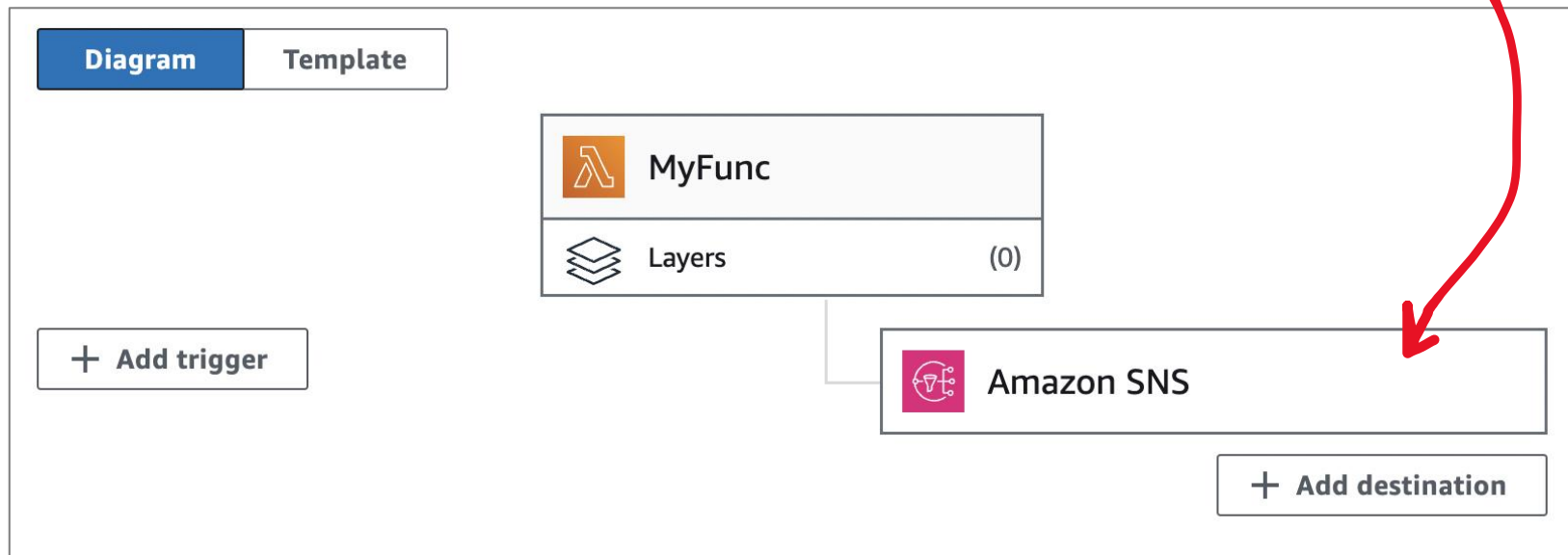
Zendesk makes better customer service experiences for agents, admins, and customers.

AWS Lambda – Destinations

AWS Lambda can be invoked in two ways:

- Sync Invocations — data the waits for a response eg. HTTP response
- Async Invocations — data the immediately returns to async AWS services

Destination is for invoking **Async invocation**



Destination:

- SNS Topic
- SQS Queue
- Lambda Function
- EventBridge Bus

Can go to destination on:

- Successful execution
- Failed execution

AWS Lambda – Memory and Timeout

Time settings:

- Default: 3 seconds
- Min: 1 second
- Max 15 minutes

Timeout [Info](#)

min sec

Storage settings:

- Default: 512 MB
- Min: 512 MB
- Max 10,240 MB (~10 GB)

Ephemeral storage [Info](#)

You can configure up to 10 GB of ephemeral s

MB

Set ephemeral storage (/tmp) to between 512

Memory settings: (Increments of 1 MB)

- Default: 128 MB
- Min: 128 B
- Max 10,240 MB (~10 GB)

Memory [Info](#)

Your function is allocated CPU proportional to the m

MB

Set memory to between 128 MB and 10240 MB

AWS Lambda – Pricing

The first **1 million requests** per month are free.
There-after **\$0.20** per additional 1 million requests

400,000 GB seconds free per month
After that, **\$0.0000166667** for every GB second

**This price will vary on the amount of memory you allocate

128MB of Memory **X** 30M executed per month **X** 200ms run time per invocation = **\$5.83**

Lambda – Function Versions

You can use **versions** to manage the deployment of your AWS Lambda functions.

eg. publish a new version of a lambda for beta testing without affecting the production version

The image consists of three screenshots from the AWS Lambda console, illustrating the process of publishing and managing function versions. Red arrows highlight key elements: the 'Publish new version' action, the 'Publish' button in the confirmation dialog, and the 'Versions' tab in the version management panel.

Left Screenshot: Actions Menu

- Actions ▲
- Select a test event
- Publish new version
- Create alias
- Export function
- Capabilities
- Deploy to Lambda@Edge
- Delete function

Middle Screenshot: Publish new version from \$LATEST

Publishing a new version saves a snapshot of the code and configuration of the \$LATEST version. You can't edit the new version's code. Click to confirm.

Version description

install self-sealing stem bolts

Cancel Publish

Right Screenshot: Switch versions/aliases

Version: \$LATEST ▲

Switch versions/aliases

Filter versions/aliases

Versions Aliases

Version	Created	Code Size	Configuration
\$LATEST (30 seconds ago)			
2 (30 seconds ago)			Realign the dilithium crystals
1 (4 minutes ago)			install self-sealing stem bolts

Each version will have its own unique ARN

- `arn:aws:lambda:us-east-1:111111111111:function: MyFunction:$LATEST`
- `arn:aws:lambda:us-east-1:111111111111:function:MyFunction:2`
- `arn:aws:lambda:us-east-1:111111111111:function:MyFunction:1`

Lambda – Function Versions

When you reference, a Lambda use its ARN.

A Lambda has two initial versions:

Qualified ARN

The function ARN **with** the version suffix.

arn:aws:lambda:aws-region:acct-id:function:helloworld:\$LATEST

Unqualified ARN

The function ARN **without** the version suffix

arn:aws:lambda:aws-region:acct-id:function:helloworld

You **cannot create Aliases with Unqualified ARN**

Unqualified ARNs **points to the Latest.**

Lambda – Aliases

Aliases allow you to give specific versions a **friendlier name** when accessing the lambda programmatically.

The image illustrates the process of creating and managing Lambda aliases through three sequential screenshots:

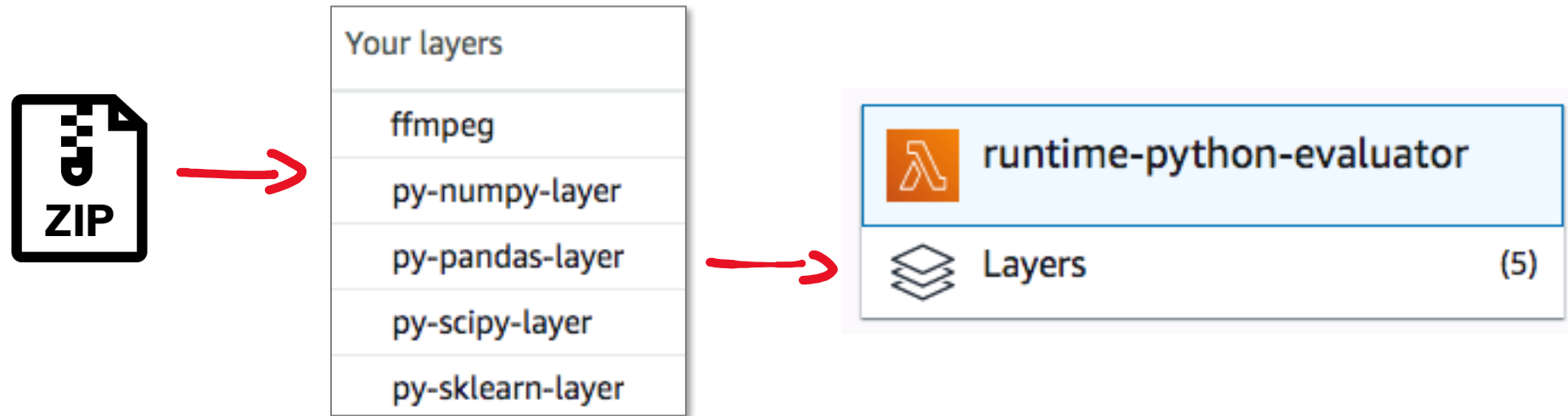
- First Screenshot:** Shows the 'Actions' menu for a Lambda function. The 'Create alias' option is highlighted with a red arrow.
- Second Screenshot:** Shows the 'Create a new alias' dialog box. The 'Name*' field is filled with 'stable-version', and the 'Version*' dropdown is set to '2'. A red arrow points from the 'Create alias' button in the first screenshot to this dialog. Another red arrow points from the 'Create' button at the bottom of the dialog to the third screenshot.
- Third Screenshot:** Shows the 'Switch versions/aliases' tab in the Lambda function details. The 'Aliases' sub-tab is selected, showing a list of aliases. The 'stable-version' alias is listed with 'Version: 2'. A red arrow points from the 'Create' button in the second screenshot to this list.

Lambda – Layers

Pull in additional code and content in the form of layers.

A layer is a ZIP archive that contains libraries, a custom runtime, or other dependencies. You can use libraries in your function without needing to include them in your deployment package

You can have up to 🖐️ **5 layers** attached to a function
all layers can't exceed the unzipped deployment **package size limit of 250 MB**.



Lambda – Instruction Set

There are two available Instruction architectures available for Lambda:

- **arm64** – 64-bit ARM architecture, for the AWS Graviton2 processor.
- **x86_64** – 64-bit x86 architecture, for x86-based processors.

An Instruction Set are a set of opcode that represent operations the CPU can perform.

- Load (LDA) — **01**x1000
- Store (STA) — **02**-1001
- Add (ADD) — **03**x1002
- Jump (JMP) — **04**x1003
- Subtract (SUB) — **05**x1004

Arm64 has a smaller instruction set therefore programs built against arm are more efficient and as a result are more cost effective.

For a program to run a specific architecture the language has to turn into a machine language such (assembly code) which in turn is calling opcodes.

When possible use Arm64 since its most proficient.

All Amazon Linux 2 (AL2) runtimes support both x86_64 and ARM CPU architectures.

Lambda – Runtimes

Lambda Runtime is a **preconfigured environment to run specific programming languages**.
Runtimes are useful since it doesn't require you to configure a container or OS configuration.
Runtimes are fully managed and are security-hardened by AWS.



Lambda Runtimes are released as stable programming languages are released.

Older Runtimes are deprecated, which will force you to upgrade your Lambdas and their code to run on more recent runtime versions.

```
aws lambda create-function \  
--function-name MyRubyFunction \  
--handler lambda_function.lambda_handler \  
--runtime ruby3.2 \  
--role arn:aws:iam::123456789012:role/execution_role \  
--zip-file fileb://function.zip
```

A runtime will specify a:

- Named Version: eg. Node.js 20
- **Identifier**: eg. nodejs20.x
- Operation System: eg. Amazon Linux 2023

The identifier is used to **tell what runtime to use**.

Code is delivered as a **ZIP archive** when using Lambda runtimes.

Lambda – OS-Only Runtimes

OS-Only Runtimes is when there is not preinstalled programming language and language specific libraries installed when you want to compile your languages or programs to be used.

There are 3 cases when to use Os-Only Runtimes:

1. Native ahead-of-time (AOT) compilation

Languages such as **Go, Rust, and C++, .NET Native AOT, Java GraalVM Native** compile natively to an executable binary, which doesn't require a dedicated language runtime.

- You must include a runtime interface client in your binary.
- You must compile your binary for a Linux environment for the same instruction set architecture

2. Third-party runtimes

- Runtimes such as Bref for PHP
- Swift AWS Lambda Runtime for Swift.

3. Custom runtimes

build your own runtime for a language or language version that Lambda doesn't provide a managed runtime.

The OS-Only Runtimes:

- Amazon Linux 2 - provided.al2023
- Amazon Linux 2023 - provided.al2

Lambda – Deployment Packages

Deployment package is the **package which contains your function code** that lambda will deploy

Lambda supports two types of deployment packages: ZIP (zip archive) and IMAGE (container image)

Zip Archive

You ZIP the contents of your code and additional programming libraries.

- AWS will upload the contents to an AWS managed S3 Bucket
- or you upload it to an S3 bucket and reference the object address to the Lambda.
 - ZIP **larger than 50 MB** have to be uploaded to an S3 bucket first.
- You rely **on Lambda runtimes** and are limited to these runtime environments.

Container Image

You build a container

- You need to create a Dockerfile
- You need to build the image
- You need to push the image to Amazon Elastic Container Registry (ECR)
- You reference the container image to Lambda

Lambda – Deployment Packages ZIP

To utilize ZIP Deployments you specify a runtime and set PackageType: **Zip**

```
AWSTemplateFormatVersion : '2010-09-09'  
Transform: AWS::Serverless-2016-10-31  
Resources:  
  InlineLambda:  
    Type: AWS::Serverless::Function  
    Properties:  
      Handler: function.handler  
      Runtime: python3.12  
      PackageType: Zip  
      CodeUri: "./function/"
```

You'll have to tell **where the archive or code to be zipped is located**: locally or an S3 address

Lambda – Deployment Packages Container

No runtime is specified.

Instead you **supply the Dockerfile**
or container image url eg. ECR

```
AWSTemplateFormatVersion : '2010-09-09'  
Transform: AWS::Serverless-2016-10-31  
Resources:  
  InlineLambda:  
    Type: AWS::Serverless::Function  
    Metadata:  
      DockerContext: "../"  
      Dockerfile: Dockerfile  
    Properties:  
      PackageType: Image
```

You set PackageType to **Image**

Lambda – Programming Model

Lambda provides a programming model that is common to all of the runtimes
The model defines the **interface between your code and the Lambda system**.

Function Handler

The **method which will be called** by Lambda

```
require 'json'

def handler(event:, context:)
  return {
    event: JSON.generate(event),
    context: JSON.generate(context.inspect)
  }
end
```

Event

event data passed from a source to the handler.
This data will greatly vary based on the source.

Context

Provides **contextual information** about the lambda.
Varies based on runtime.

The “**handler**” not to be confused with the function handler is specified to Lambda to tell it what file and function should be called

A handler of “lambda_function.lambda_handler” for ruby

- Expects a file called **lambda_function.rb**
- Expects a function in the file called **lambda_handler**

Lambda – Programming Model

Node.Js

```
exports.handler = async (event) => {  
  const response = {  
    statusCode: 200,  
    body: JSON.stringify('Hello World'),  
  };  
  return response;  
};
```

Python

```
def lambda_handler(event, context):  
    return {  
        'statusCode': 200,  
        'body': 'Hello World'  
    }
```

PowerShell

```
function Handler {  
    param($LambdaInput, $Context)  
    return "Hello World"  
}  
Export-ModuleMember -Function Handler
```

Ruby

```
def lambda_handler(event:, context:)  
  { statusCode: 200, body: 'Hello World' }  
end
```

Java

```
package example;  
  
import com.amazonaws.services.lambda.runtime.Context;  
import com.amazonaws.services.lambda.runtime.RequestHandler;  
  
public class Handler implements RequestHandler<Object, String> {  
    @Override  
    public String handleRequest(Object input, Context context) {  
        return "Hello World";  
    }  
}
```

Lambda – Programming Model

C# .NET

```
using Amazon.Lambda.Core;
using Amazon.Lambda.Serialization.Json;

namespace HelloWorld
{
    class LambdaHandler
    {
        [LambdaSerializer(typeof(JsonSerializer))]
        public string handleRequest(object evt, ILambdaContext context)
        {
            return "Hello World";
        }
    }
}
```

Rust

```
use lambda_http::{lambda, IntoResponse, Request};
use lambda_runtime::{Error, Context};
use log::LevelFilter;
use simple_logger::SimpleLogger;

async fn hello_world(_request: Request, _context: Context) -> Result<impl IntoResponse, Error> {
    Ok("Hello World")
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    SimpleLogger::new().with_level(LevelFilter::Info).init().unwrap();
    lambda!(hello_world);
    Ok(())
}
```

Golang

```
package main

import (
    "github.com/aws/aws-lambda-go/lambda"
)

func HandleRequest(ctx context.Context,
    event *MyEvent (string, error) {
    return "Hello World", nil
}

func main() {
    lambda.Start(HandleRequest)
}
```

Lambda – Programming Model

Event Data passed the to function handler **greatly varies between sources**.
When you manually invoke a Lambda function via the AWS API you can provide any json.

Event data from **Amazon S3**

```
{
  "Records": [{
    "s3": {
      "bucket": {
        "name": "my-bucket"
      },
      "object": {
        "key": "my-object"
      }
    },
    "eventTime": "2020-01-01T12:00:00.000Z",
    "eventName": "ObjectCreated:Put"
  }]
}
```

Event data from **API Gateway**

```
{
  "httpMethod": "POST",
  "path": "/myresource",
  "queryStringParameters": {
    "foo": "bar"
  },
  "headers": {
    "Content-Type": "application/json"
  },
  "body": "{\"test\":\"body\"}",
  "isBase64Encoded": false
}
```

Custom event data from a
manual invocation via the
AWS API

```
{
  "hello": "world",
  "fruits": ["apple", "banana", "orange"]
}
```

Lambda – Programming Model

Context provides information about the Lambda's runtimes and configuration. Data returned varies between Lambda runtimes

Common data returned by all runtimes:

- **functionName** — The name of the Lambda function.
- **functionVersion** — The version of the function being executed.

invokedFunctionArn

- The Amazon Resource Name (ARN) used to invoke the function.
- This can be useful for identifying the AWS account and function version or alias.

memoryLimitInMB

- The amount of memory allocated to the function.

awsRequestId

- A unique identifier for the request. This can be useful for correlating logs and tracing.

logGroupName

- The CloudWatch Logs group name associated with the function's logs.

logStreamName

- The CloudWatch Logs stream name for the function's logs.

identity

- (only for Amazon Cognito triggered functions)



```
{
  "function_name": "my_lambda_function",
  "function_version": "$LATEST",
  "invoked_function_arn": "arn:aws:lambda:us-west-2:123456789012:function:my_lambda_function",
  "memory_limit_in_mb": "128",
  "aws_request_id": "843f90-some-request-id",
  "log_group_name": "/aws/lambda/my_lambda_function",
  "log_stream_name": "2023/03/15/[$LATEST]843f90someLogStream",
  "deadline_ms": 1234567890,
  "identity": {
    "cognito_identity_id": "US-SOME-ID",
    "cognito_identity_pool_id": "US-SOME-POOL-ID"
  },
  "client_context": {
    "client": {
      "installation_id": "some-installation-id",
      "app_title": "My App",
      "app_version_name": "1.0",
      "app_version_code": "1",
      "app_package_name": "com.example.myapp"
    },
    "custom": {},
    "env": {
      "platform_version": "5.1.1",
      "platform": "Android",
      "make": "Samsung",
      "model": "S5"
    }
  }
}
```

Lambda – Code Signing

Code signing for AWS Lambda helps to ensure that **only trusted code runs in your Lambda functions**. Lambda **checks every code deployment** and verifies that the code package is signed by a trusted source.



You create a **AWS Signer signing profile** using a certificate in ACM

```
aws signer put-signing-profile \
--platform-id "AWSLambda-SHA384-ECDSA" \
--profile-name "MySigningProfile" \
--signing-material certificateArn="..." \
--signing-parameters platform="AWSLambda"
```

Then you create a **Lambda code Signing Config** referencing your AWS Signer profile

```
aws lambda create-code-signing-config \
--description "My code signing config for Lambda" \
--allowed-publishers signingProfileVersionArns="..." \
--code-signing-policies untrustedArtifactOnDeployment="ENFORCE"
```

Then **attach** the Lambda Code Signing Config to the Lambda

```
aws lambda update-function-code-signing-config \
--function-name "MyLambdaFunction" \
--code-signing-config-arn "..."
```

The user must then sign their deployment package when uploading.

```
aws signer start-signing-job \
--source "s3={bucketName=source-bucket,bucketKey=function.zip,bucketVersion=1}" \
--destination "s3={bucketName=dest-bucket}" \
--profile-name YourProfileName
```

Lambda – Function Blueprints

Blueprints are **language-specific samples that include function code and settings** for a runtime and trigger combination.

Blueprints are available for **Node.js** and **Python** runtimes.

Blueprint name	
Process CloudWatch log data A real-time consumer of log events ingested by an Amazon CloudWatch Logs log group.	nodejs18.x ▲
🔍 Search by blueprint name, runtime or workload category	
Returns the current status of an AWS Batch Job.	
Make an appointment with Lex Schedule a dentist appointment, using Amazon Lex to perform natural language understanding	python3.10
Manage orders with Lex Order flowers, using Amazon Lex to perform natural language understanding	python3.10
Modify Cognito records based on an event An Amazon Cognito Sync trigger that modifies Cognito datasets based on the incoming event.	nodejs18.x
Process CloudWatch log data A real-time consumer of log events ingested by an Amazon CloudWatch Logs log group.	nodejs18.x ✓
Send CloudWatch alarm notifications via SNS An Amazon SNS trigger that sends CloudWatch alarm notifications to Slack.	python3.10
Send CloudWatch logs to Loggly Sends logs from Cloudwatch logs to Loggly using a Lambda function.	nodejs18.x
Send Kinesis Analytics records to custom destination Deliver output records from Kinesis Analytics application to custom destination.	nodejs18.x
Submit Batch job Submits an AWS Batch Job and returns the jobId.	python3.10
Submit Batch job	nodejs18.x

Lambda Applications

AWS Lambda application is a way of publishing CFN or SAM templates to quickly launch serverless workloads.

Lambda Applications integrate with:

- CodePipeline for **CI/CD**
- Codebuild for **Build and Test**
- Code Commit or Github for **Source Code**
- CloudFormation for **Deployment**

Lambda Applications are different from Lambda Blueprints:

- Blueprints is for a single Lambda
- Lambda Applications is a collection of AWS Services with tight coupling with services to serve an app eg. CodePipeline, CodeBuild

[Lambda](#) > [Applications](#) > Review, configure and deploy

ffmpeg-lambda-layer — version 1.0.0

Review, configure and deploy

Application details

Author	Source code URL	Description
Gojko Adzic 	https://github.com/serverlesspub/ffmpeg-aws-lambda-layer 	Static build of FFmpeg/FFprobe for Amazon Linux 2, packaged as a Lambda Layer. Version: FFmpeg 4.1.3

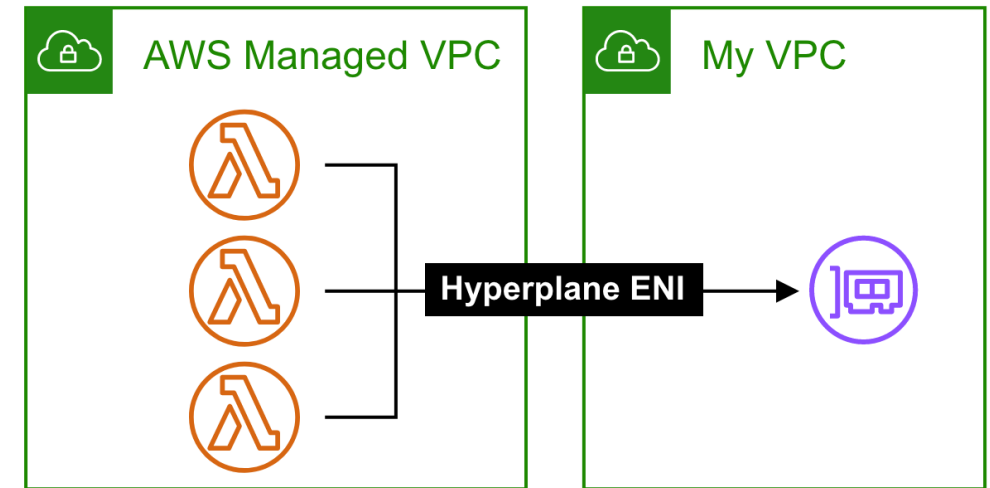
▼ Template

```
AWSTemplateFormatVersion: '2010-09-09'
Description: |
  Static build of FFmpeg/FFprobe for Amazon Linux 2
  Check out https://github.com/serverlesspub/ffmpeg-aws-lambda-layer
Outputs:
  LayerVersion:
    Description: Layer ARN Reference
    Value:
      Ref: LambdaLayer
Resources:
  LambdaLayer:
    Properties:
```

Lambda – Private Networking

Lambda functions **by default run within AWS managed VPC** and Lambdas do not have access to your VPCs. Lambdas can **be deployed into your VPCs** for when Lambdas needs to communicate with your VPC resources.

- Lambda accesses resources in your VPC using a Hyperplane ENI
- Hyperplane ENIs provide NAT capabilities from the Lambda VPC to your account VPC using VPC-to-VPC NAT (V2N)
- V2N is one direction.
 - Lambda VPC to your account VPC
 - but not in the other direction.
- For each subnet, Lambda creates a network interface for each unique set of security groups.
- Functions in the account that share the same subnet and security group will use the same Elastic Network Interface (ENI).



To **configure a Lambda to deploy to a VPC** you set the Subnets and Security Groups →

```
--vpc-config \  
SubnetIds=subnet-0123456789abcdef0,subnet-0f1e2d3c4b5a6978,\  
SecurityGroupIds=sg-085912345678492fb
```

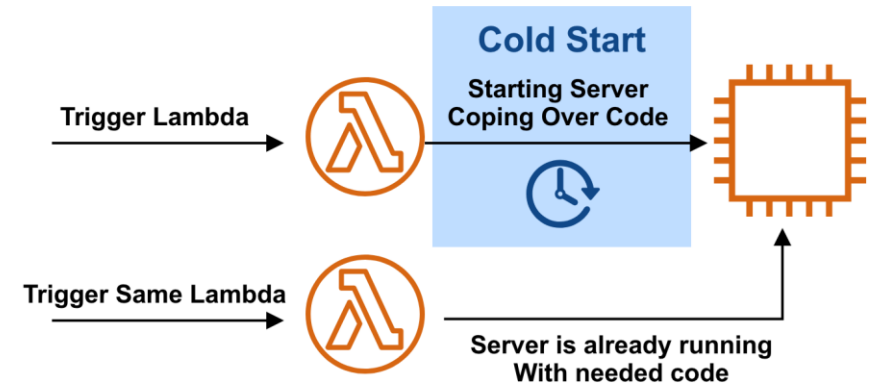
There is no additional cost to deploying a Lambda into a VPC or utilizing a Hyperplane ENI

AWS Lambda - Cold Starts

AWS has servers preconfigured (just sitting around turned off) for your runtime environment. When a Lambda is invoked, these servers need to be turned on, and your code needs to be copied over.

During this time, there will be a **delay when the function initially runs** which is called a **Cold Start**

If the same Lambda is invoked and the server is still running, it will use that server again, so there will be little to no delay in running that function. This is what we call a **Warm Server**



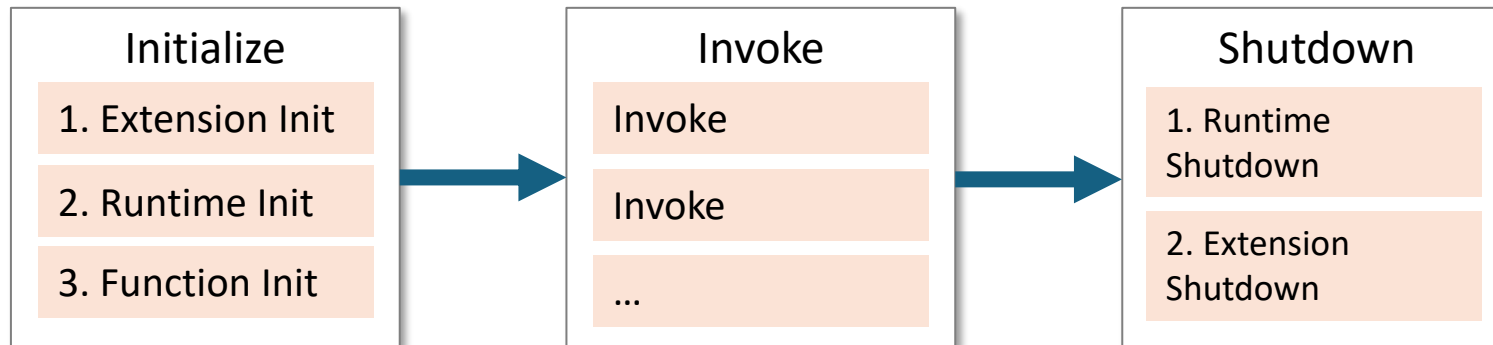
Serverless functions are **cheap**, but everything comes with a trade-off. Serverless functions Cold Starts can **cause delays in the User Experience**. If your web application relies on being very responsive, you should reconsider Serverless functions.



There are strategies around Cold Starts, such as **Pre-warming**, which keep servers continuously running. Cloud Providers are always looking for ways to reduce cold starts.

Lambda – Execution Environment

The Lifecycle of Lambda execution has three stages: Initialize, Invoke and Shutdown



Init phase

- 1. Start all extensions
- 2. Bootstrap the runtime
- 3. Run the function's static code
- 4. Run any beforeCheckpoint runtime hooks (SnapStart only)
- ends when the runtime and all extensions signal that they are ready by sending a Next API request
- phase is limited to 10 seconds
 - If first three tasks fail to complete in 10 seconds, Lambda retries

Invoke phase

- Ready to accept Invoke API calls to run the lambda
- The Invoke phase is limited to function timeout
- ends when the runtime and all extensions signal that they are ready by sending a Next API request

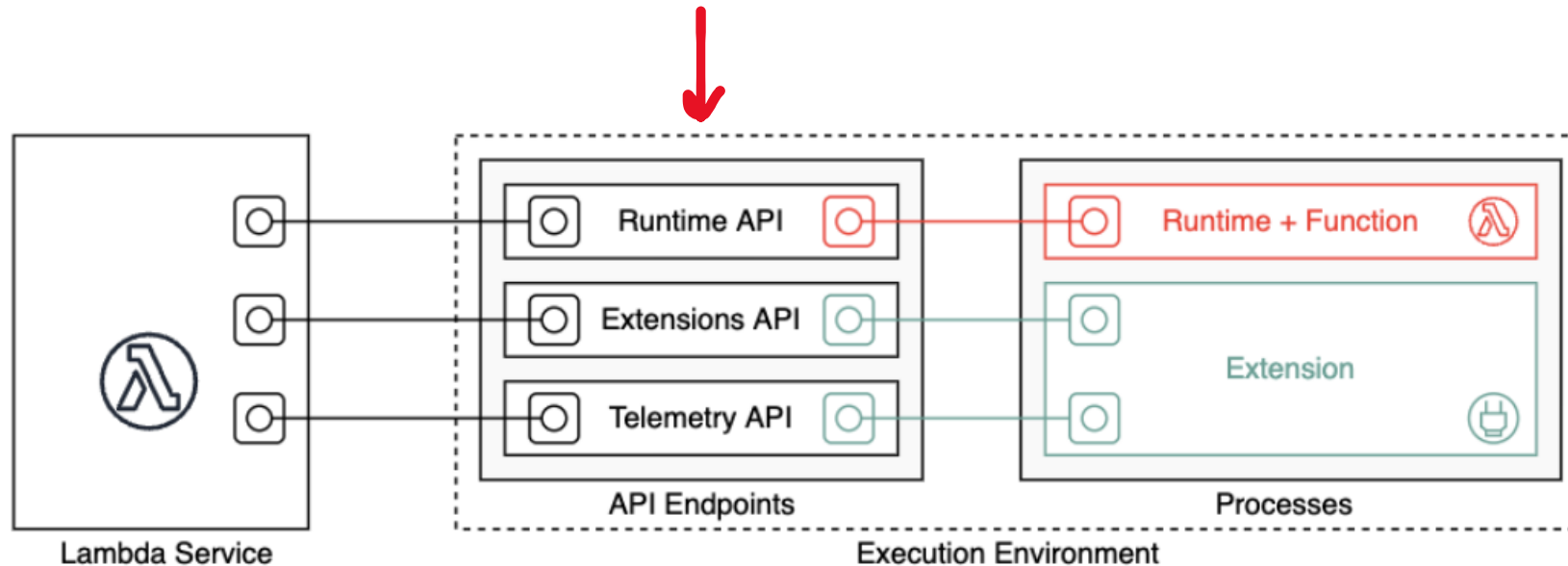
Shutdown phase

- sends a Shutdown event to each registered external extension
- Shutdown phase is capped at 2 second
- If the runtime or any extension does not respond, Lambda terminates it via a signal (SIGKILL).

Lambda – Execution Environment

The execution environment contains two API Endpoints:

- **Runtime API** — this is used to run or build custom runtimes
- **Extensions API** — this is used to run or build extensions which integrations to tool for monitoring, observability, security or governance tools
- **Telemetry API** — this enables your extensions to receive telemetry data directly from Lambda.

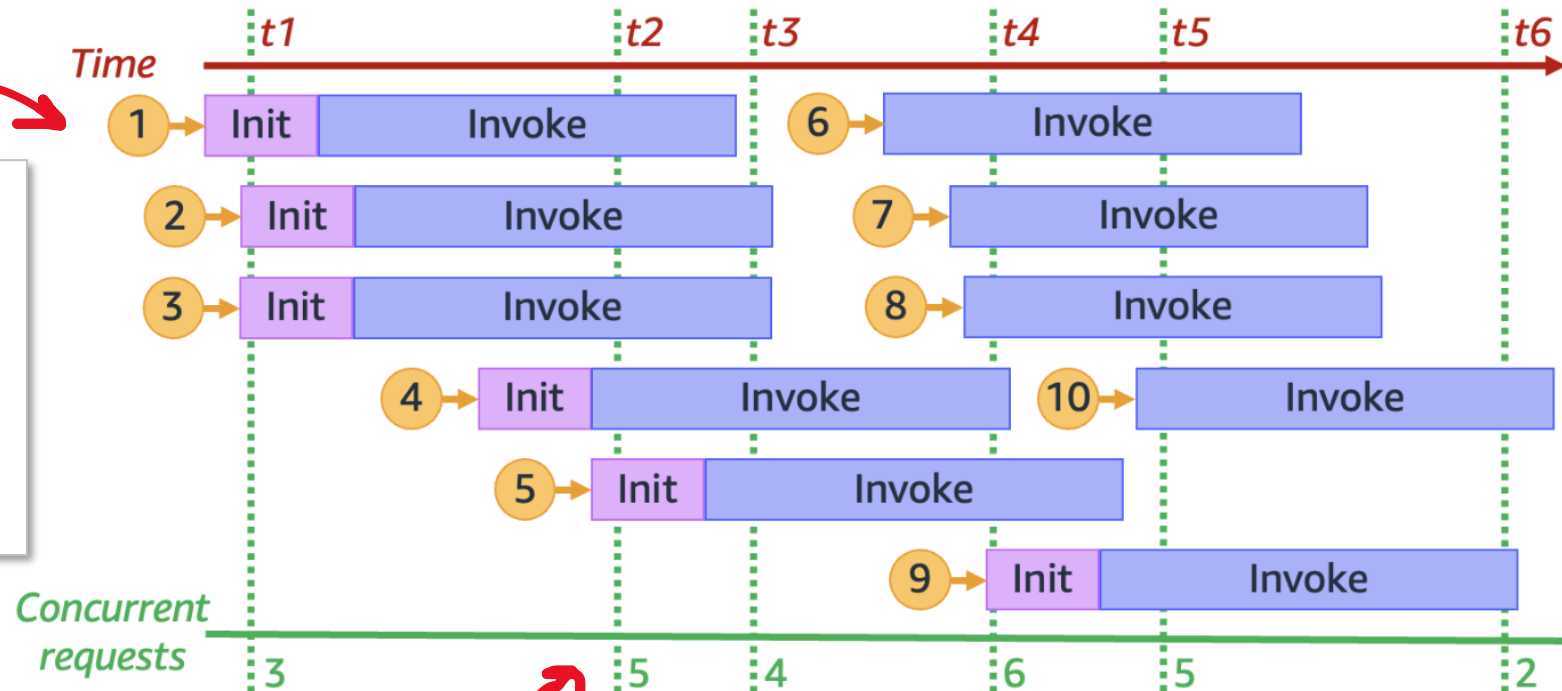


Lambda – Concurrency

For each concurrent request, **Lambda provisions a separate instance of your execution environment.** By default, Lambda provides your account with a total **concurrency limit of 1,000** concurrent executions across all functions in an AWS Region

The amount of compute instances available

When a Lambda's task is complete another incoming request will use that compute, otherwise it will provision a new instance to concurrently run multiple instances.



The amount of computes instances currently running (concurrently running)

Lambda – Reserved Concurrency

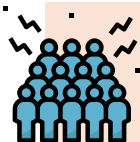
Reserved Concurrency allows you reserve from your regional concurrency pool a **guarantee of available instances** for a specific function.

Settings the reserved capacity for a specific Lambda via **AWS CLI**

```
aws lambda put-function-concurrency \
  --function-name my-function \
  --reserved-concurrent-executions 50
```

The default concurrency pool per region is 1,000

Provisioned concurrency will be used before it starts to use the Reserved concurrency



Since Lambda functions draw from the same concurrency pool a less than important Lambda could consume all the concurrency instances causing unavailability for a mission critical Lambda function.

Lambda – Provisioned Concurrency

Provisioned Concurrency is a **pool of compute instances that are prewarmed** and ready to execute your Lambda function code to **ensure low-latency and avoiding cold-starts**.

When you provision concurrent for a lambda you **have to specific a Lambda Version or Lambda Alias** as the qualifier.



```
aws lambda put-provisioned-concurrency-config \  
--function-name my-function \  
--qualifier 1 \  
--provisioned-concurrent-executions 5
```

The amount of concurrency you can provisioned is the amount of unreserved concurrency subtracting 100 per region.

- You've 500 reserved concurrency
- You have 500 unreserved concurrency remaining
- Subtracting 100 you have 400 available provisioned concurrency

Lambda – Provisioned vs Reserved Concurrency

	Reserved Concurrency	Provisioned Concurrency
Main Purpose	Guarantees availability and limits scale of a function.	Eliminates cold starts and ensures immediate response from a function.
Concurrency Allocation	Sets max of concurrent executions for a function.	Pre-initializes a specified number of execution env.
Impact on Cold Starts	Does not directly affect cold starts.	Significantly reduces or eliminates cold starts.
Billing	No additional costs beyond standard Lambda usage fees.	Incurs an ongoing cost for all provisioned instances
Use Case	Ensuring a min level of service for critical functions. Preventing a function from consuming too many resources.	Applications requiring consistent, low-latency responses. High-traffic events where performance is critical.
Scalability	Limits function scalability to manage resources and costs.	Ensures scalable and immediate response capability up to the provisioned amount.
Management	Can be set per function to manage scale and impact on resources.	Requires planning and management to align with demand and control costs.
Ideal for	• Functions that must always be available. • Managing shared resources among multiple functions.	• Latency-sensitive applications. • Ensuring high availability and responsiveness during peak times.

Lambda – Environment Variables

Lambda Environment Variables (Env Vars) allows you to **set and pass environment variables** to the runtime environment.

Setting Lambda Env Vars via **AWS CLI**

```
aws lambda update-function-configuration \  
--function-name my-function \  
--environment "Variables={BUCKET=my-bucket,KEY=file.txt}"
```

```
def lambda_handler(event:, context:)  
    # Access environment variables  
    bucket_name = ENV['BUCKET']  
    key = ENV['KEY']  
    # ...  
end
```

Lambda – Environment Variables

Lambda predefines the following Env Vars

- **_HANDLER**: Specifies the handler location configured for the Lambda function.
- **_X_AMZN_TRACE_ID**: Contains the X-Ray tracing header, which changes with each invocation. It's not defined for OS-only runtimes or Java runtime versions 17 and later, where Lambda uses a system property instead.
- **AWS_DEFAULT_REGION** and **AWS_REGION**: Specify the default AWS Region and the AWS Region of the Lambda function execution, respectively. If both are defined, **AWS_REGION** overrides **AWS_DEFAULT_REGION**.
- **AWS_EXECUTION_ENV**: Identifies the runtime environment. Not defined for OS-only runtimes.
- **AWS_LAMBDA_FUNCTION_NAME**: The name of the Lambda function.
- **AWS_LAMBDA_FUNCTION_MEMORY_SIZE**: The amount of memory allocated to the Lambda function in MB.
- **AWS_LAMBDA_FUNCTION_VERSION**: The version of the function being executed.
- **AWS_LAMBDA_INITIALIZATION_TYPE**: Indicates the function's initialization type (on-demand, provisioned-concurrency, or snap-start).
- **AWS_LAMBDA_LOG_GROUP_NAME** and **AWS_LAMBDA_LOG_STREAM_NAME**: Specify the CloudWatch Logs group and stream names for the function. Not available in Lambda SnapStart functions.
- **AWS_ACCESS_KEY**, **AWS_ACCESS_KEY_ID**, **AWS_SECRET_ACCESS_KEY**, **AWS_SESSION_TOKEN**: Access keys from the function's execution role.
- **AWS_LAMBDA_RUNTIME_API**: For custom runtimes, the host and port of the runtime API.
- **LAMBDA_TASK_ROOT**: The file system path to the Lambda function code.
- **LAMBDA_RUNTIME_DIR**: The file system path to runtime libraries.

Lambda – Temporary Storage

AWS Lambda provides **temporary storage of files** generated or stored within the runtime only during the lifetime of the runtime located at **/tmp**

Changing amount of temporary storage via **AWS CLI**

```
aws lambda update-function-configuration \  
--function-name PDFGenerator \  
--ephemeral-storage '{"Size": 10240}'
```

Ephemeral storage:

- Default: 500 MB
- Min: 500 MB
- Max: 10 GB

To persist data you can:

- mount EFS
- Upload files to Amazon S3

Lambda – Mounting Elastic File System

AWS Lambda function can be configured to mount an Elastic File System (EFS) volume.

Lambda storage is ephemeral where as mounting EFS **allows the persistence of files between Lambdas** and other compute resources eg. Fargate, EC2.

CFN mounting EFS to a lambda at **/mnt/efs**

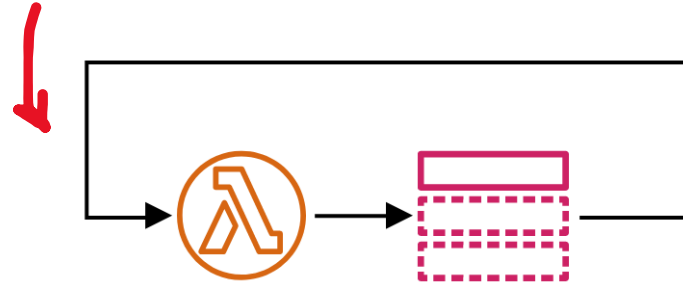


```
MyFunctionWithEfs:
  Type: AWS::Serverless::Function
  Properties:
    Handler: index.handler
    Runtime: python3.10
    VpcConfig:
      SecurityGroupIds:
        - Ref: EfsSecurityGroup
      SubnetIds:
        - Ref: Subnet1
    FileSystemConfigs:
      - Arn: !GetAtt AccessPoint.Arn
        LocalMountPath: "/mnt/efs"
    Description: Use a file system.
```

Lambda Execution role will need to be updated to allow access to EFS

Lambda – Recursive Loop Detection

Lambda recursive loop detection prevents a Lambda being called in an **indefinite loop** and resulting in unexpected spend.



Lambda recursive loop detection work with:

- Simple Queue Service (SQS)
- Simple Notification Service (SNS)
- Directly using the Lambda invoke API.

Lambda detects functions that appear to be running in a recursive loop and drops requests **after exceeding 16 invocations**

- When a recursive loop is detected you'll receive an email from AWS that its been stopped
 - Reach out to AWS support to resolve the issue
- There is no additional charge for this feature.
- This feature is turned on by default to stop recursive loops.
- To request turning it off for your account, reach out to AWS Support.

Lambda – Function URLs

Lambda Function URL is a **dedicated HTTP(S) endpoint** to invoke your Lambda function.

<https://<url-id>.lambda-url.<region>.on.aws>

You can set the Auth Type to be:

NONE — Anyone can invoke the lambda via the URL

IAM — HTTP request must be signed by an IAM principle using AWS SigV4, a tool like awscurl can automate this process.

CORS can be configure to further limit what and who can invoke the function

```
aws lambda create-function-url-config \
  --function-name my-function \
  --auth-type NONE \
  --cors '{
    "AllowCredentials": false,
    "AllowHeaders": ["x-example-header"],
    "AllowMethods": ["POST", "GET"],
    "AllowOrigins": ["https://example.com"],
    "ExposeHeaders": ["x-example-header"],
    "MaxAge": 3600
  }'
```



Lambda Function URLs are not supported in some regions.

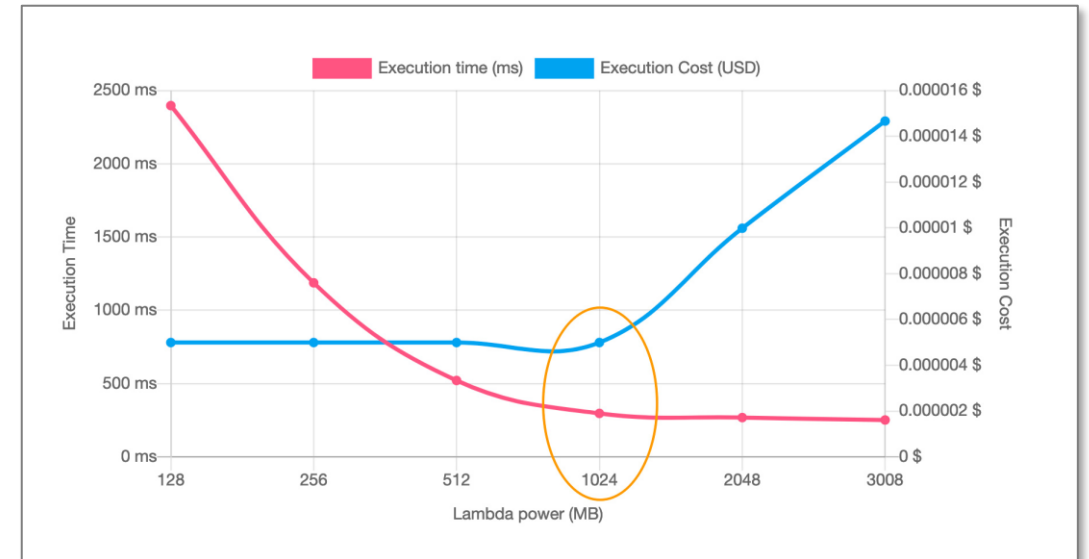
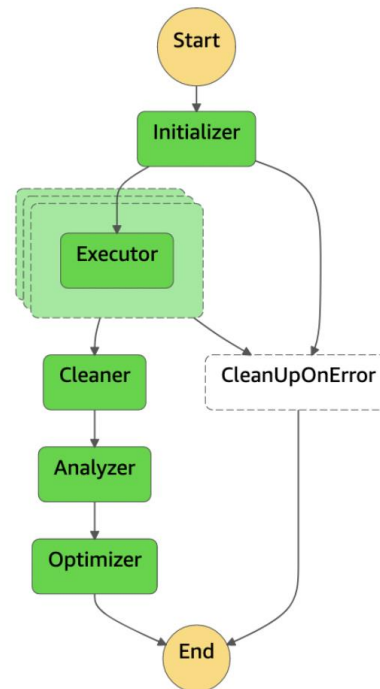
Lambda – Power Tuning

AWS Lambda Power Tuning is an open-source profiling tool to help you profile an existing function to rightsize a Lambda tuning the memory and duration parameters.

<https://github.com/alexcasalboni/aws-lambda-power-tuning>

AWS Lambda Power Tuning will utilize Step Functions to run a specific lambda, tuning the parameters each run and then providing raw data.

The data can be visualized into a graph so you can then determine optimal parameter settings.



Lambda – Execution Role

Lambda execution role is an IAM role that grants the function permission to AWS services and AWS resources

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "*"
    }
  ]
}
```



AWS has the following managed policies for Lambda:

- AWSLambdaMSKExecutionRole
- **AWSLambdaBasicExecutionRole**
- AWSLambdaDynamoDBExecutionRole
- AWSLambdaKinesisExecutionRole
- AWSLambdaMSKExecutionRole
- AWSLambdaSQSQueueExecutionRole
- AWSLambdaVPCLambdaAccessExecutionRole
- AWSXRayDaemonWriteAccess
- CloudWatchLambdaInsightsExecutionRolePolicy
- AmazonS3ObjectLambdaExecutionRolePolicy

Lambda – Function Logs

Lambda will **automatically write logs to CloudWatch Logs**
(if your execution role has permission)

Within your Lambda functions you can send logs in either:

- Plain text
- Stringify JSON
 - JSON will be formatted within logs.
 - JSON logs can be easily analyzed

The following logging libraries/methods are supported:

- **Java** - the LambdaLogger logger or Log4j2.
- **Node.js** - The console methods console.trace, console.debug, console.log, console.info, console.error, and console.warn
- **Python** - the standard Python logging library
- For all other runtimes, writing to STDOUT should be captured within the logs.

```
def lambda_handler(event:, context:)  
    # will log plain text  
    puts "Write this to cloudwatch logs"  
    # we log json  
    puts JSON.generate(:action => "hello")  
    return {  
        statusCode: 200,  
        body: JSON.generate('Hello World')  
    }  
end
```



Example of **ruby** logging to cloudwatch using puts (prints) to STDOUT

Lambda Extensions

Lambda Extensions is a way to **integrate Lambda with monitoring, observability, security, and governance tools.**

Extension use-cases:

- capturing diagnostic information before, during, and after function invocation
- automatically instrumenting your code without needing code changes
- fetching configuration settings or secrets before the function invocation
- detecting and alerting on function activity through hardened security agents, which can run as separate processes from the function



Internal Extensions:

- runs as an independent process in the execution environment and continues to run after the function invocation is fully processed.



External Extensions:

- run as separate processes, you can write them in a different language than the function.

Extensions are installed via packaged zip archives uploaded to **Lambda Layers.**

Lambda Extensions

Third-Party Managed Extensions:

- AppDynamics
- Check Point CloudGuard
- Datadog
- Dynatrace
- Elastic
- Epsagon
- Fastly
- HashiCorp Vault
- Honeycomb
- Lumigo
- New Relic
- Sedai
- Sentry
- Site24x7
- Splunk
- Sumo Logic
- Thundra
- Salt Security

AWS Managed extensions:

- AWS AppConfig
- Amazon CodeGuru Profiler
- CloudWatch Lambda Insights
- AWS Distro for OpenTelemetry (ADOT)
- AWS Parameters and Secrets

Lambda SnapStart

Lambda SnapStart for Java can **improve startup performance for latency-sensitive applications** by up to 10x at no extra cost, with no changes to your function code.



Lambda SnapStart takes a **Firecracker microVM** snapshot of the memory and disk state of the initialized execution environment, encrypts the snapshot, and caches it for low-latency access.

There's no additional cost for SnapStart.



SnapStart supports Java 11 and later Java managed runtimes.

SnapStart does not support:

- Provisioned concurrency
- The arm64 architecture
- Amazon Elastic File System (Amazon EFS)
- ephemeral storage greater than 512 MB.

Activate SnapStart via the **AWS CLI**

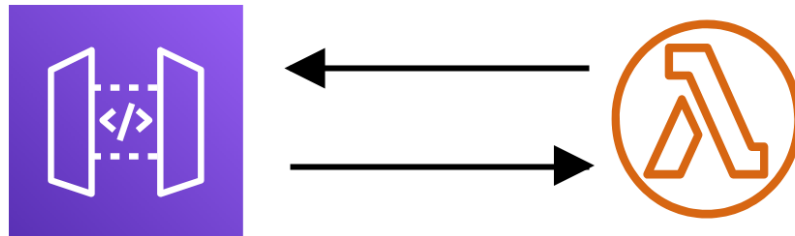
```
aws lambda update-function-configuration \  
--function-name my-function \  
--snap-start ApplyOn=PublishedVersions
```

Lambda Sync Invocation

Synchronous invocation is when a Lambda is **directly invoked by the Invoke API** by a user or a AWS computing service and expects an immediate response returned.

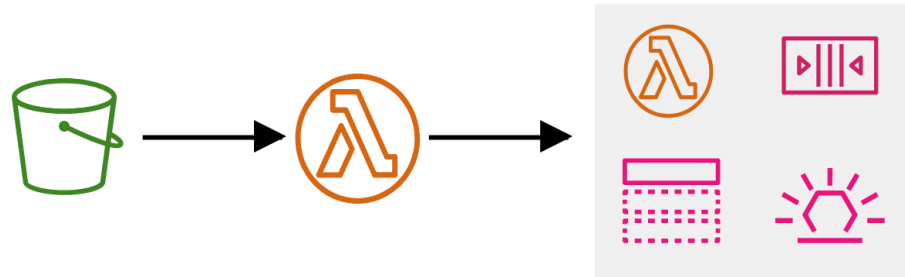
```
aws lambda invoke \  
--function-name my-function \  
--cli-binary-format raw-in-base64-out \  
--payload '{ "key": "value" }' response.json
```

An example of an AWS Service using Sync Invocation is API Gateway which expects an immediate HTTP response



Lambda Async Invocation

Asynchronous invocation is when a Lambda is invoked by event data and you send event data and response payload after Lambda execution to an AWS messaging system (Lambda Destinations) eg: SNS, SQS, EventBridge or another Lambda.



Async Invocation allows for building serverless, lightweight and low-code event-driven architecture.

Adding a destination to another AWS Lambda after successful execution

```
aws lambda put-function-event-invoke-config \
  --function-name MySourceFunction \
  --destination-config '{
    "OnSuccess":{
      "Destination":"arn:aws:lambda:us-east-1:123456789012:function:MySuccessHandler"
    }
  }'
```

Lambda Sync vs Async Invocation

	Sync	Async
Invocation	Waits for response	Immediate return; processes in background
Use Case	Immediate processing needed (e.g., APIs)	Delayed processing acceptable (e.g., uploads).
Response	Caller receives immediate response	Output directed to AWS services, not caller
Error Handling	Caller handles errors based on response	AWS Lambda retries, with DLQ for unhandled errors
Examples	AWS API Gateway, AWS SDK	S3 notifications, SNS, CloudWatch Events
Concurrency	Caller manages demand peaks	AWS Lambda manages scaling automatically
Costs	Based on request count and execution time.	Same, but without waiting for response
Source	Triggered by user actions/request.	Triggered by AWS services/custom apps
Feedback	Immediate, for interactive apps	No immediate feedback, suits event-driven apps
Timeouts	Subject to caller's timeouts	Only Lambda's timeout applies